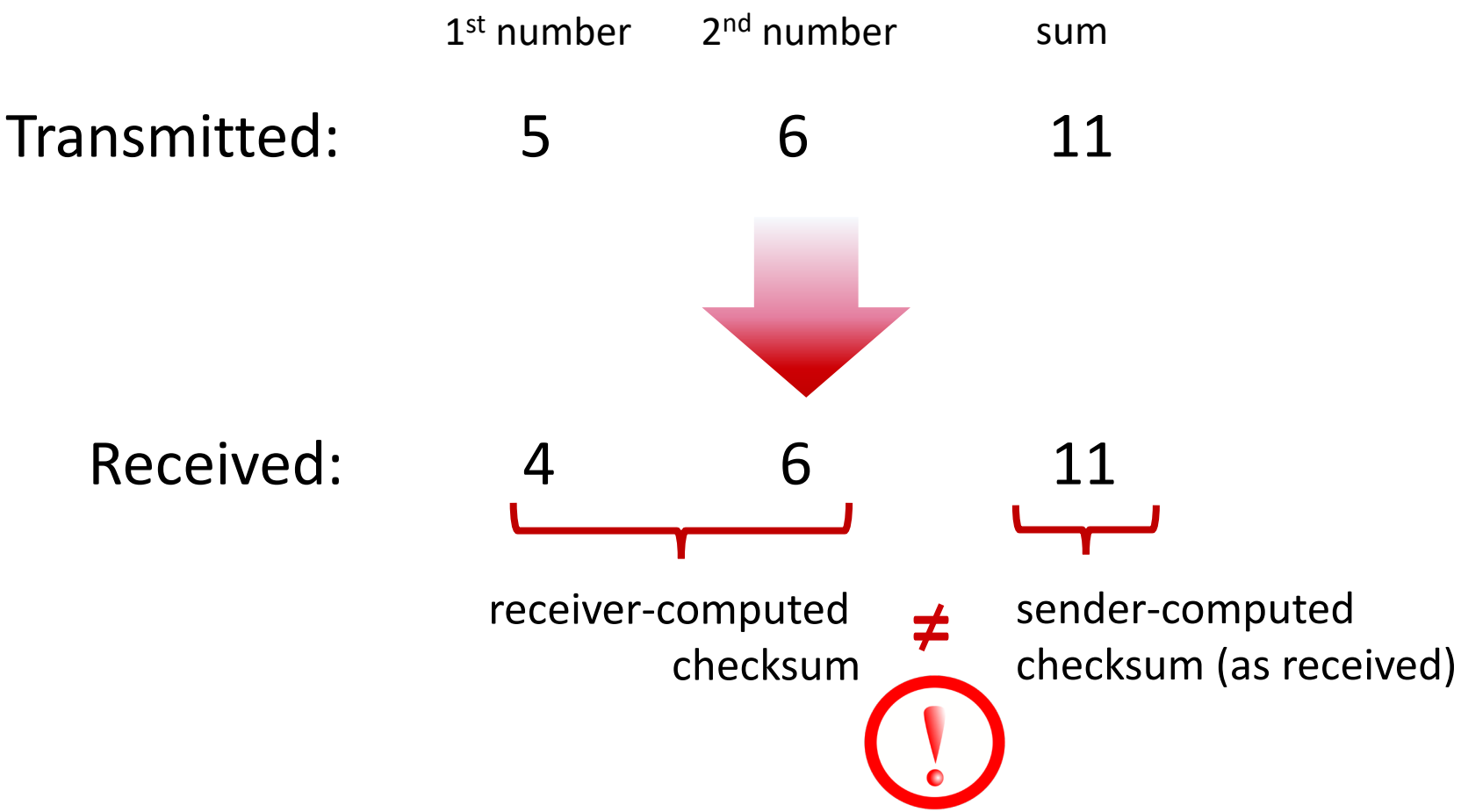
Lecture 12

- UDP Checksum
- Principles of Reliable Data Transfer (RDT)

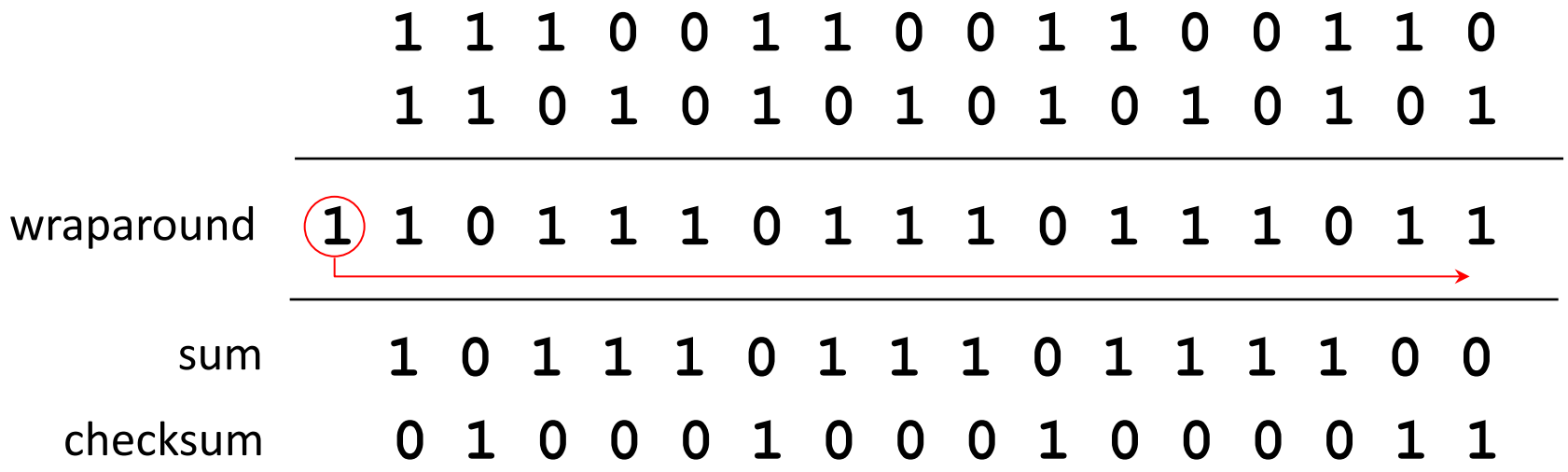
UDP checksum

Goal: detect errors (*i.e.*, flipped bits) in transmitted segment



UDP checksum: Sender

- Treat contents of UDP segment (*including UDP header fields*) as sequence of 16-bit integers
- Checksum: addition (one's complement sum) of segment content
- Checksum value put into UDP checksum field



- **Note:** when adding numbers, a carryout from the most significant bit needs to be added to the result

UDP checksum: Receiver

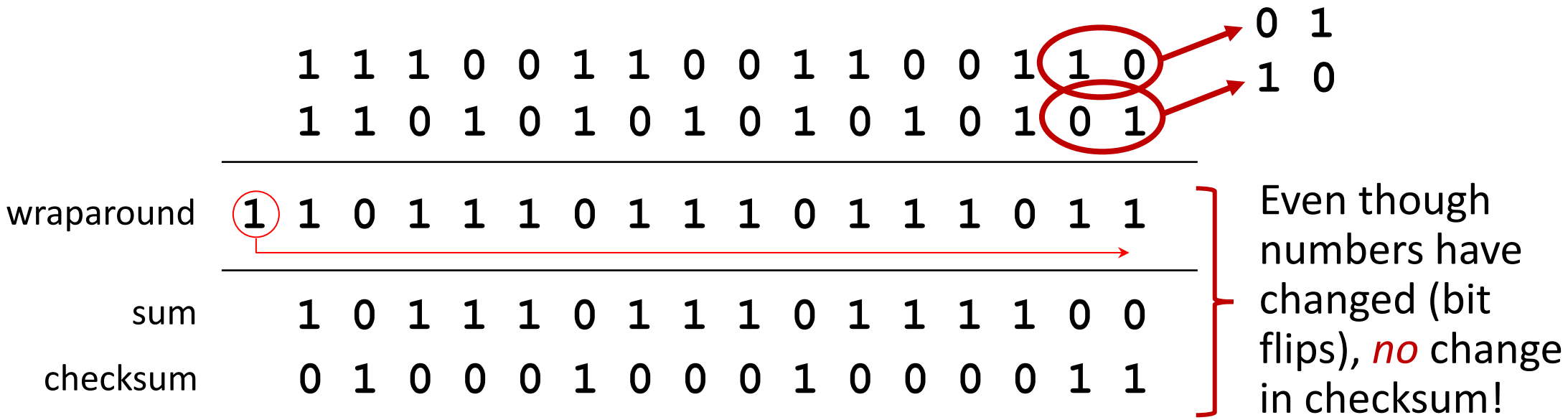
- Compute checksum of received segment
- check if computed checksum equals checksum field value:
  - Not equal - error detected
  - Equal - no error detected. *But maybe errors nonetheless?* More later ….

	1	1	1	0	0	1	1	0	0	1	1	0	0	1	1	0	**A**
	1	1	0	1	0	1	0	1	0	1	0	1	0	1	0	1	**B**
wraparound	(1)	1	0	1	1	1	0	1	1	1	0	1	1	1	0	1	1	
sum	1	0	1	1	1	0	1	1	1	0	1	1	1	1	0	0	
checksum	0	1	0	0	0	1	0	0	0	1	0	0	0	0	1	1	**C**

- In practice: receiver calculates the value A+B+C
- If it is ***not*** equal to 1111111111111111 ➔ error detected.
- The sender chooses the checksum so that **data + checksum = all 1s (0xFFFF)** under one's-complement arithmetic.

Internet checksum: weak protection!

example: add two 16-bit integers



Summary: UDP

- “best effort” protocol:
  - segments may be lost, delivered out of order
  - best effort service: “send and hope for the best”
- UDP has its plusses:
  - no setup/handshaking needed (no RTT incurred)
  - can function when network service is compromised/congested
  - Some preliminary checksum implementation
- If needed, we can add additional functionality on top of UDP in application layer

Principles of Reliable Data Transfer

Principles of reliable data transfer (RDT)

Goals

1. Data Integrity
   - No packet loss
   - No packet duplicated
   - In-order delivery
2. Flow control
   - Avoid overflow on receiver's buffer

Mechanisms

- Acknowledgment (ACK)
- Time-out
- Retransmission

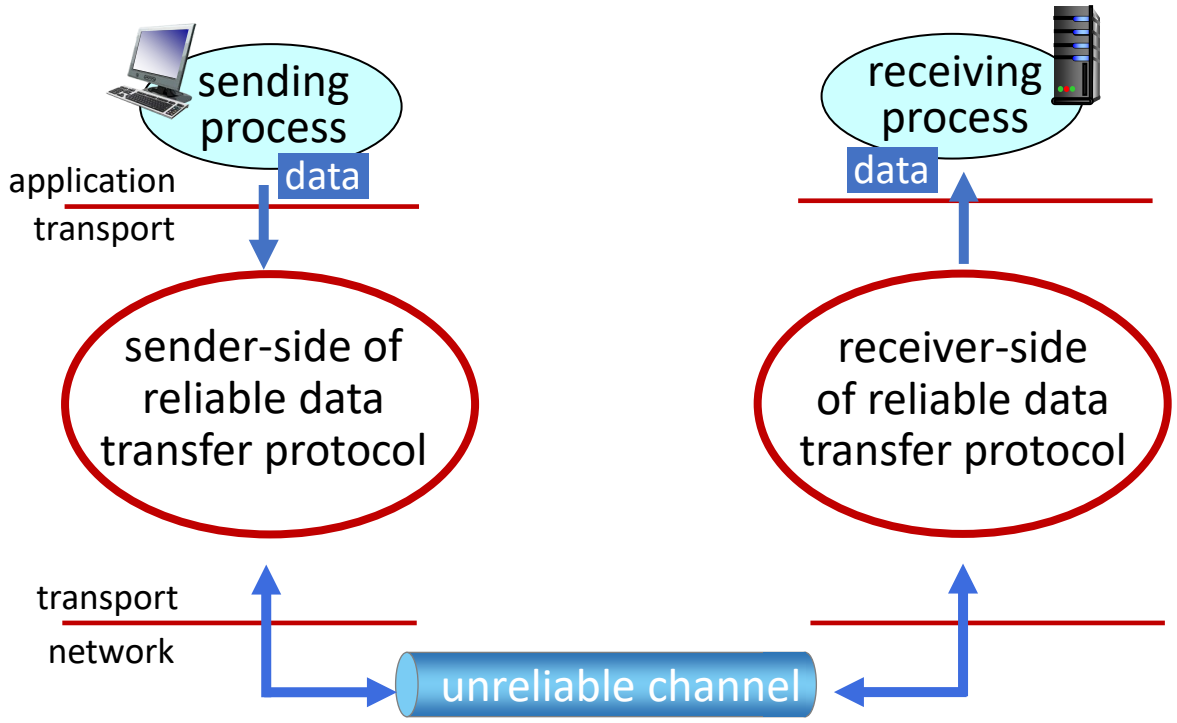
Principles of reliable data transfer



reliable service *abstraction*

Principles of reliable data transfer

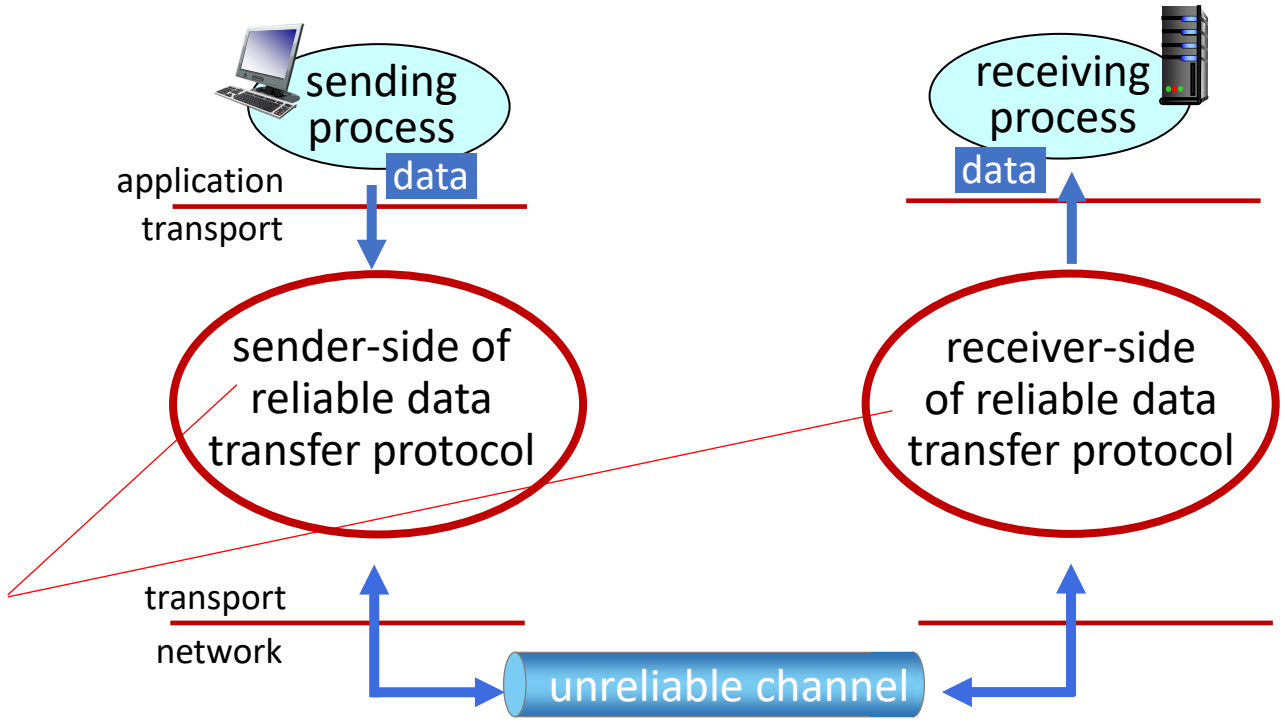
reliable service *abstraction*



reliable service *implementation*

Principles of reliable data transfer

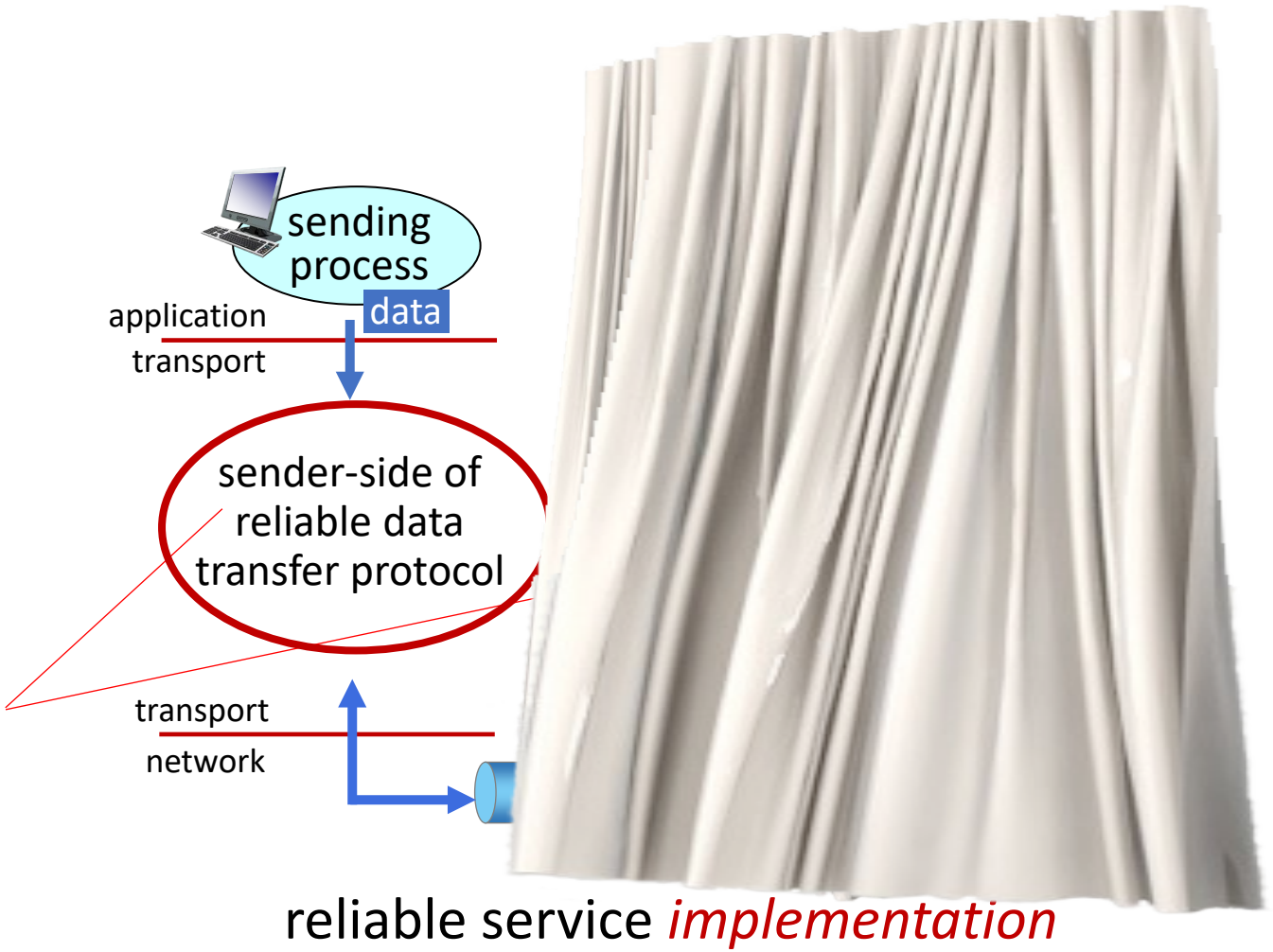
Remark 1: Complexity of reliable data transfer protocol will depend on the characteristics of unreliable channel (loss, corrupt, reorder data?)



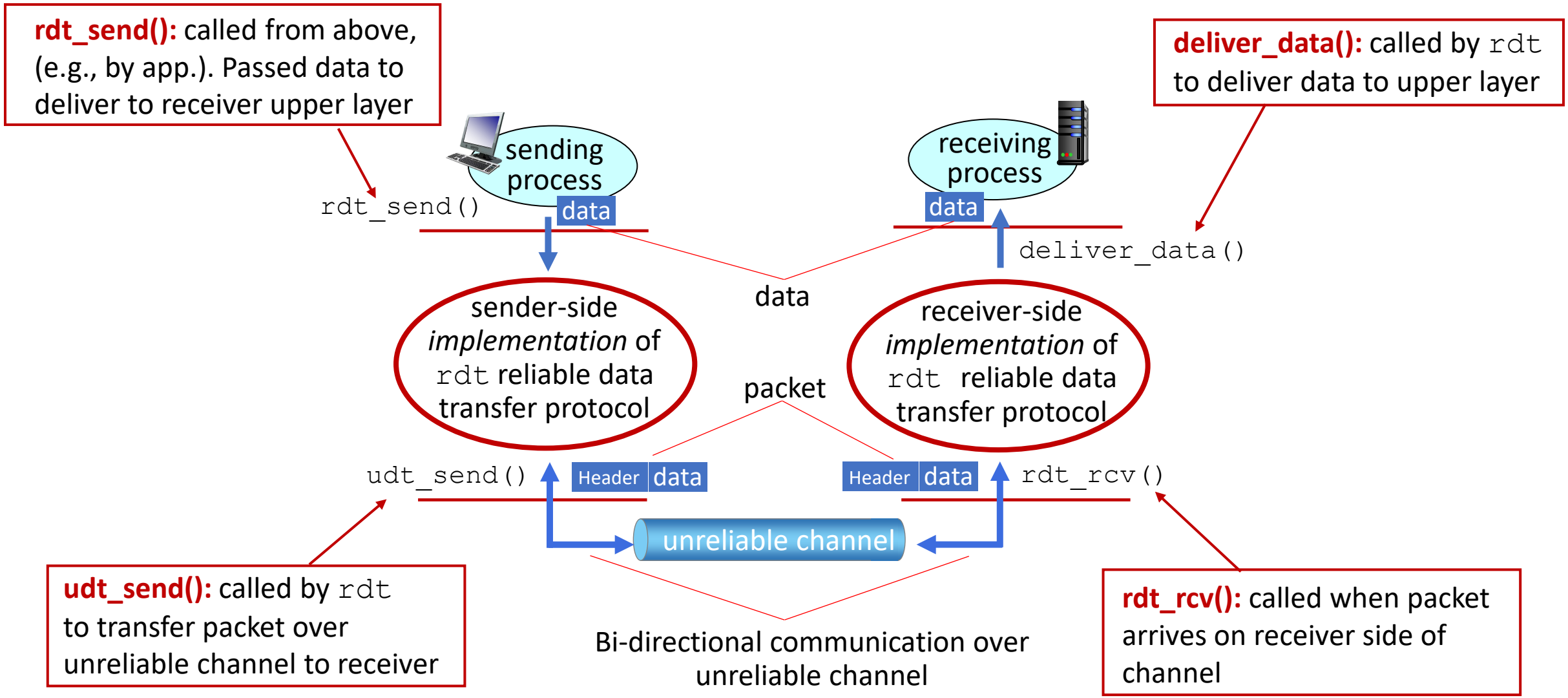
reliable service *implementation*

Principles of reliable data transfer

Remark 2: Sender, receiver do ***not*** know the "state" of each other, e.g., was a message received? **unless communicated via a message!**

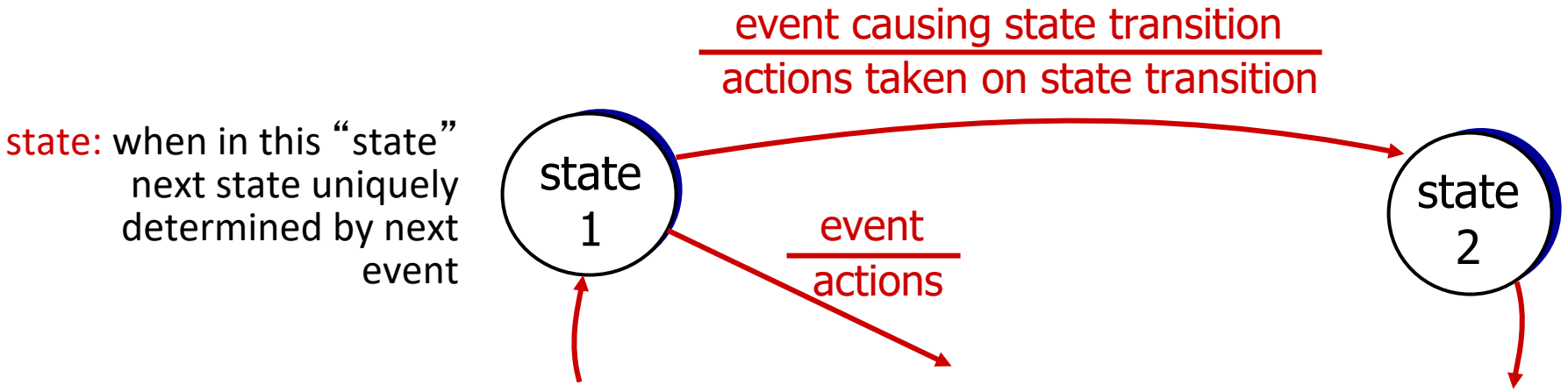


Reliable data transfer protocol (rdt): interfaces



Reliable data transfer: Finite State Machine (FSM)

- Use finite state machines (FSM) to specify sender, receiver

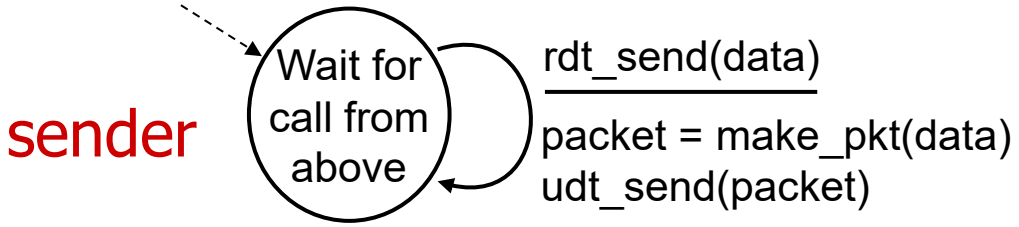


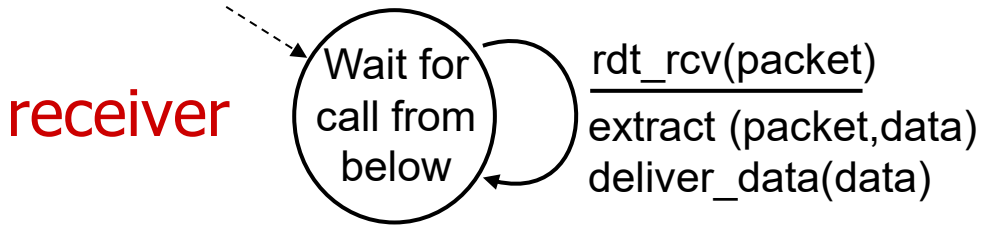
- ***Incrementally develop*** sender, receiver sides of reliable data transfer protocol (`rdt`)

rdt1.0: reliable transfer over a reliable channel

- underlying channel perfectly reliable
  - no bit errors
  - no loss of packets
- *separate* FSMs for sender, receiver:







Designing rdt2.0

- **Scenario:** Alice wants to send packets to Bob over an unreliable channel.
  - The channel may **corrupt bits (e.g., 0 to 1), but packets are never lost.**
  - Alice ***cannot*** directly see what happens at Bob.
- **Question:** Design the minimum mechanisms needed so that Bob always receives the correct data.
- **Think about:**
  - How can Bob determine whether a packet is corrupted?
  - How can Bob tell Alice what happened?
  - What should Alice do if something went wrong?

rdt2.0: channel with bit errors

- Underlying channel may flip bits in packet ➔ Packet error
- **Question 1:** how to detect errors at the receiver?
  - checksum (e.g., Internet checksum) to detect bit errors
- **Question 2:** how to recover from errors?

How do humans recover from "errors" during conversation?

rdt2.0: channel with bit errors

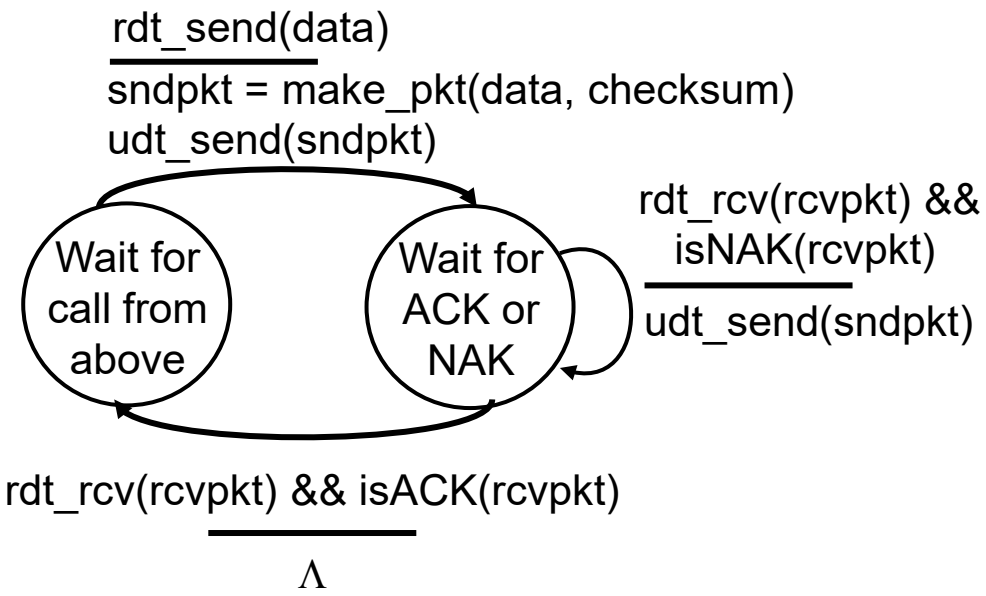
- How to recover from errors?
  - *Acknowledgements (ACKs):* receiver explicitly tells sender that pkt received OK
  - *Negative acknowledgements (NAKs):* receiver explicitly tells sender that pkt had errors
  - Sender *retransmits* pkt on receipt of NAK

stop and wait

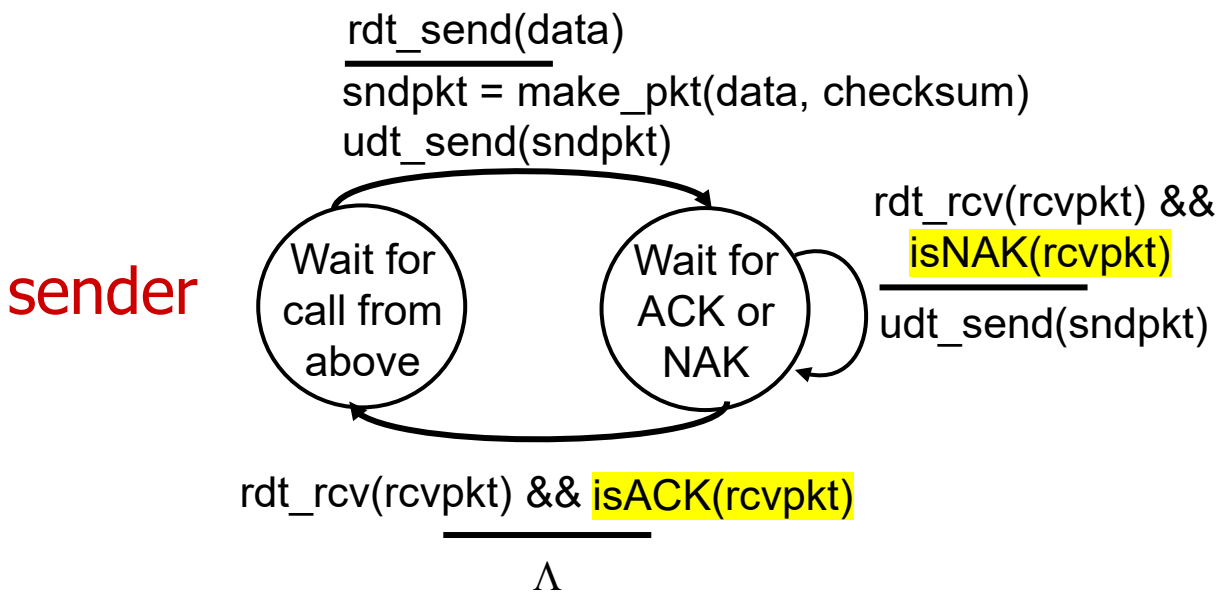
sender sends one packet, then waits for receiver response

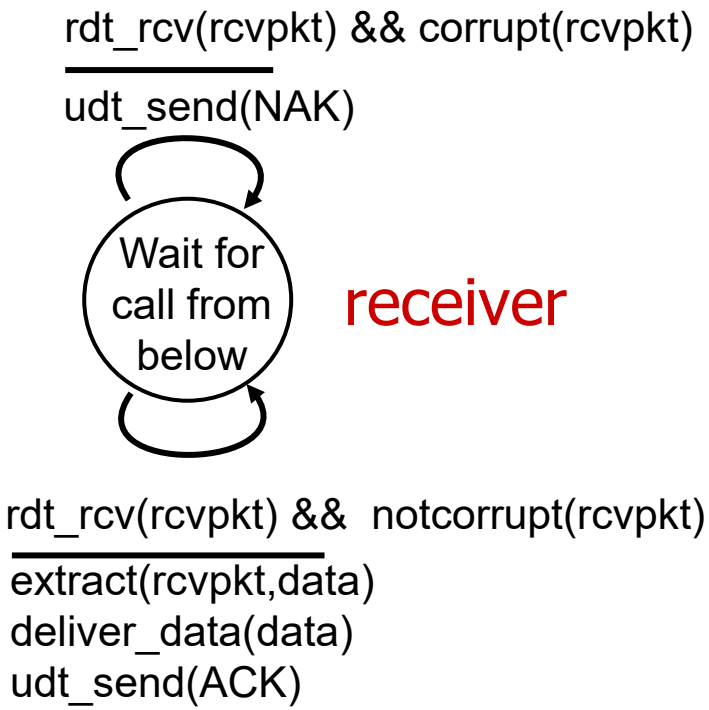
rdt2.0: FSM specification

sender



rdt2.0: FSM specification





Discussion: Problem with rdt2.0?

What if the ACK or NAK itself gets corrupted?

- Suppose the receiver actually got the packet correctly and sent ACK, but the ACK was corrupted.
- **One approach**: Alice retransmits the same packet.
- *How does Bob know this isn't new data?*

Discussion: Problem with rdt2.0?

- The sender retransmits a packet because its ACK was corrupted. What additional information allows the receiver to recognize that the retransmitted packet is a duplicate?

A. Checksum
B. Timeout
C. Packet ID
D. Larger packet size

Discussion: Problem with rdt2.0?

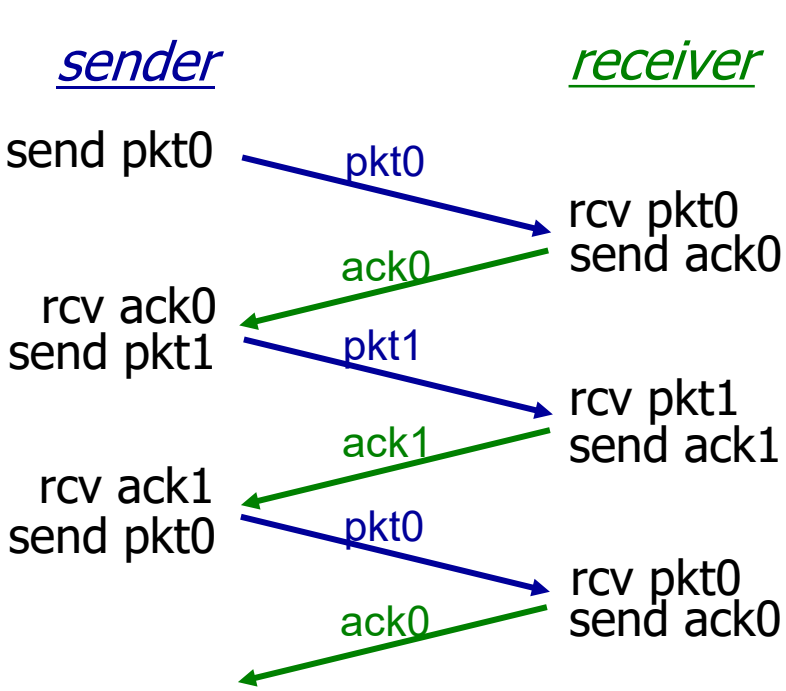
What happens if ACK/NAK corrupted?

- sender doesn't know what happened at receiver!
- can't just retransmit: possible duplicate

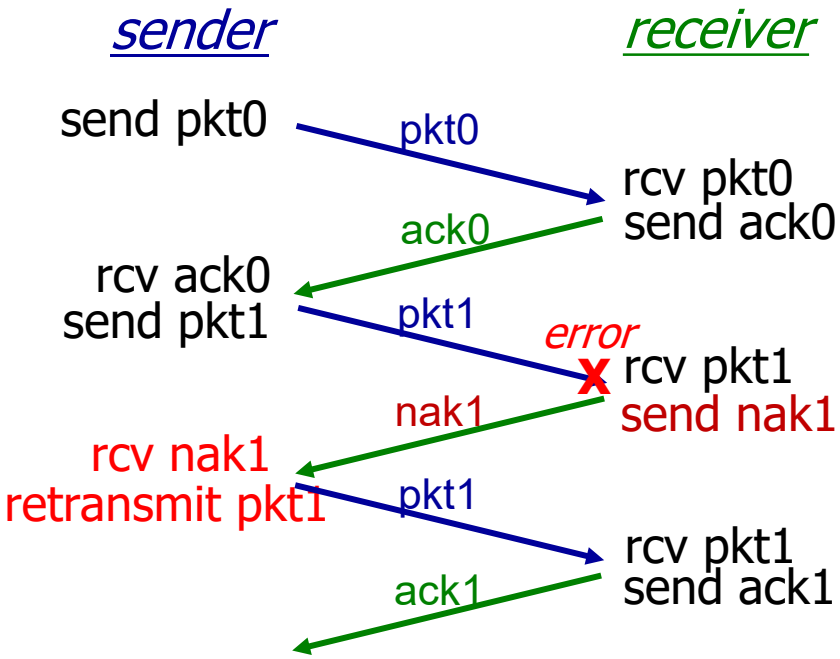
Managing duplicates:

- sender retransmits current pkt if ACK/NAK corrupted
- sender adds *sequence number* to each pkt
- receiver discards (doesn't deliver up) duplicate pkt

Stop and Wait in action

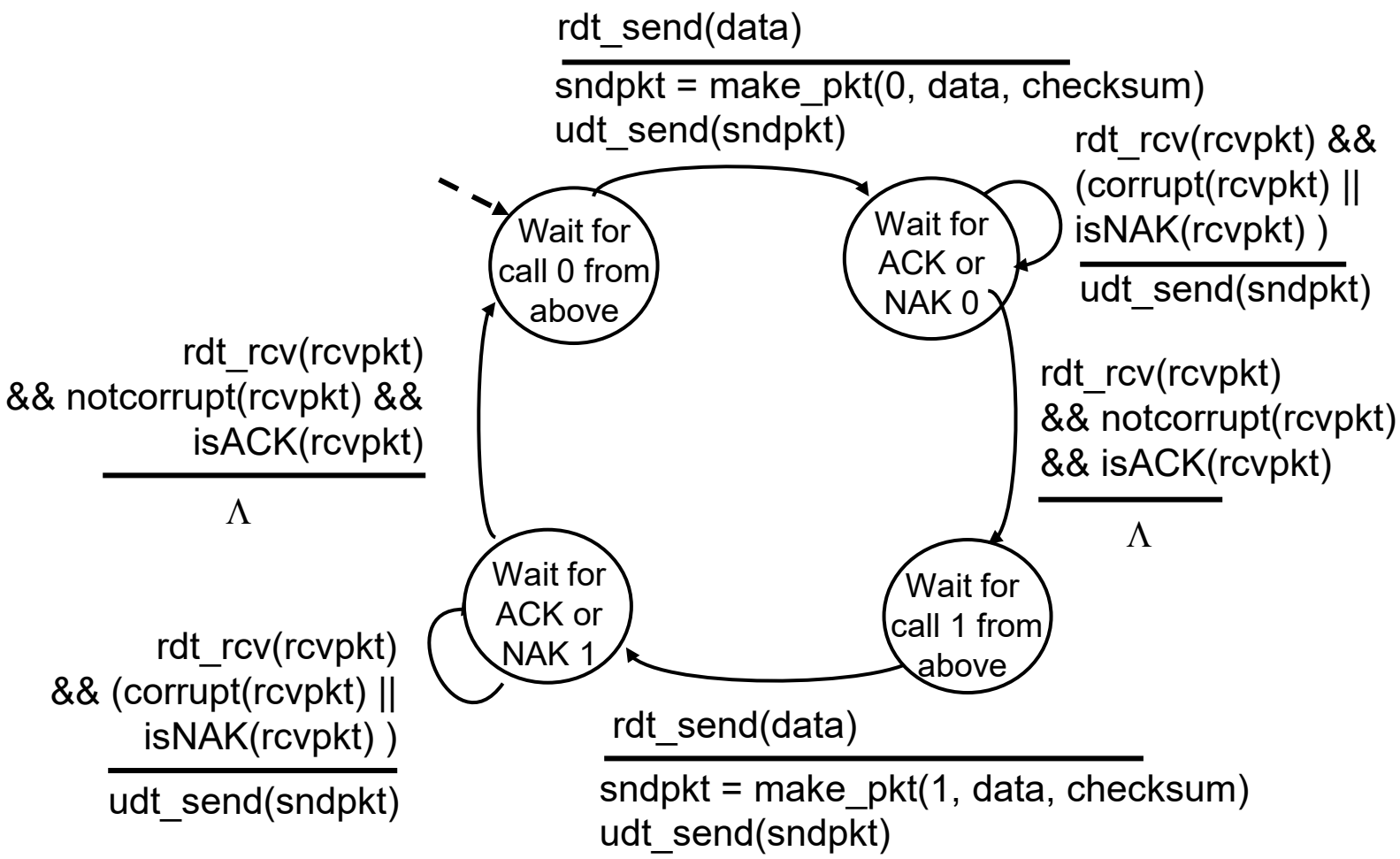


(a) No error

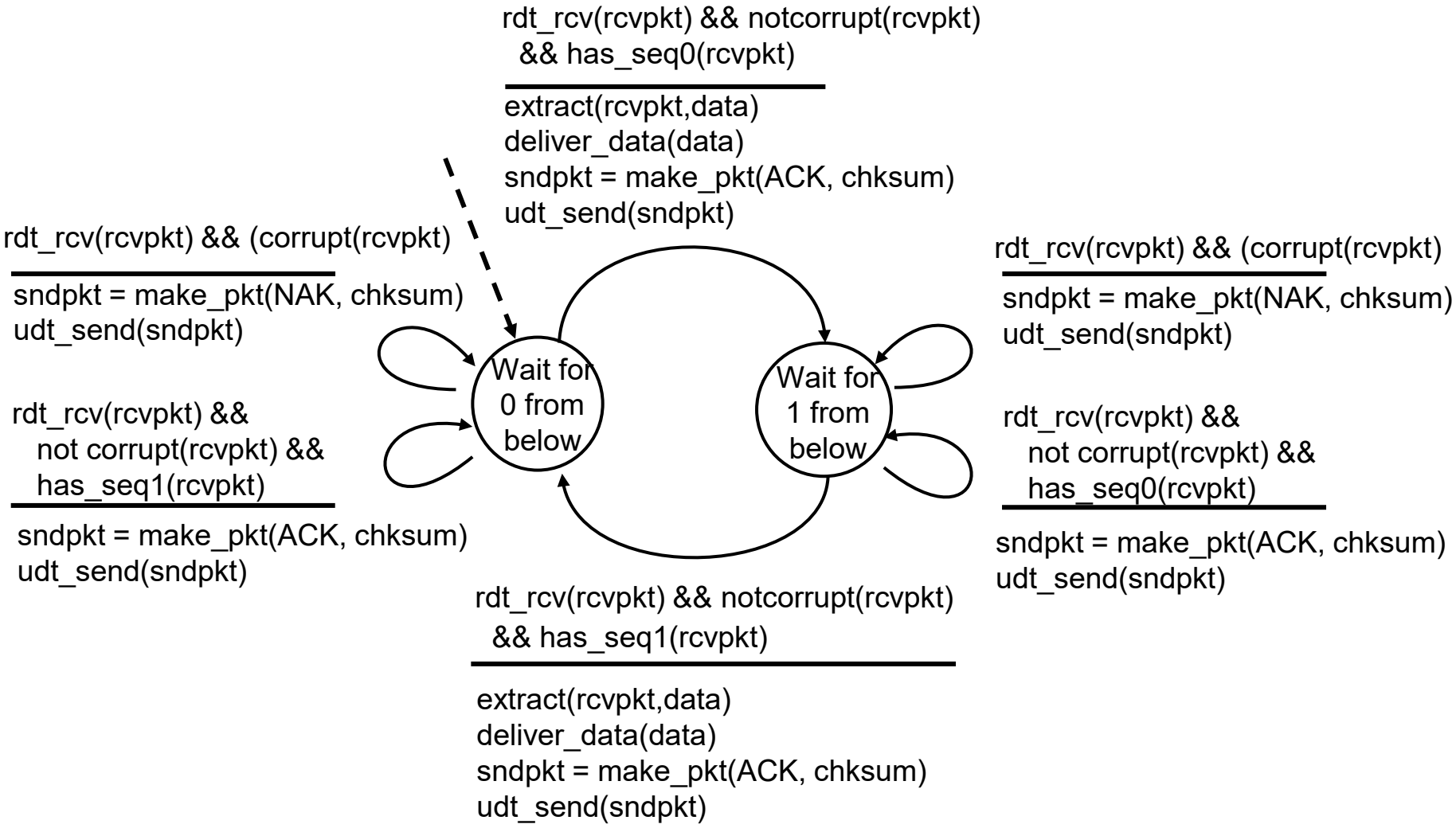


(b) packet error

rdt2.1: Sender



rdt2.1: Receiver

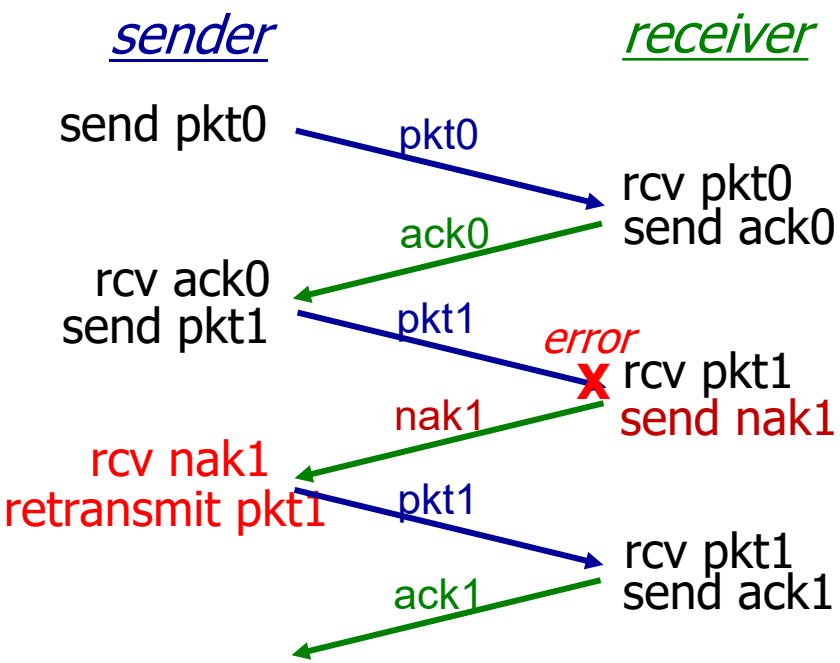


rdt2.2: a NAK-free protocol

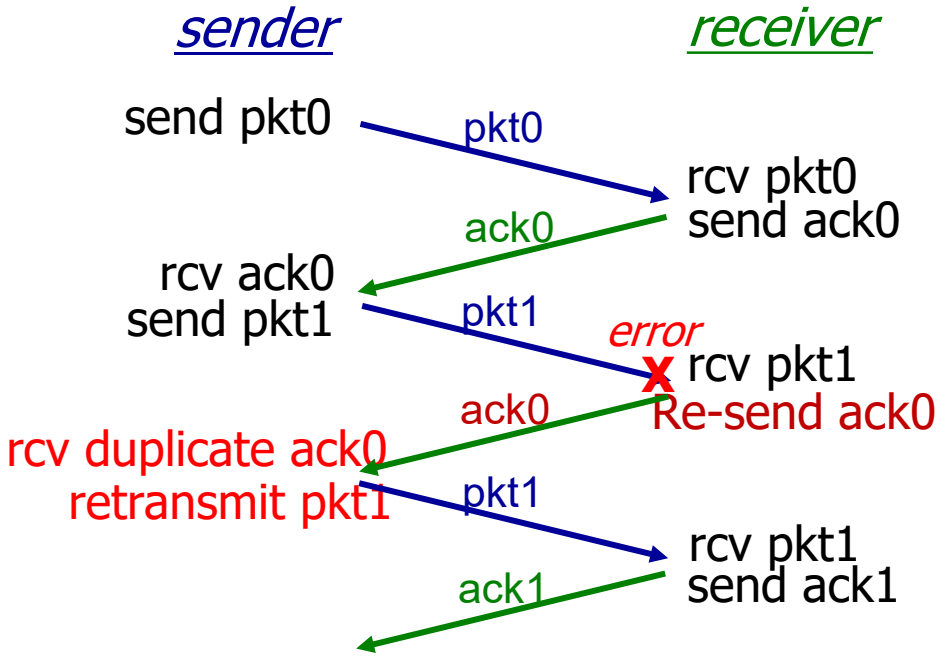
- same functionality as rdt2.1, using ACKs only
- instead of NAK, receiver sends ACK for last pkt received OK
  - receiver must *explicitly* include seq # of pkt being ACKed
- duplicate ACK at sender results in same action as NAK: *retransmit current pkt*

As we will see, TCP uses this approach to be NAK-free.

rdt2.2: a NAK-free protocol



(a) packet error – with NAK protocol



(b) Packet error – NAK free protocol

rdt3.0: channels with errors *and* loss

New channel assumption: underlying channel can also *lose* packets (data, ACKs)

- checksum, sequence #s, ACKs, retransmissions will be of help … but **not quite enough**

Q: How do *humans* handle lost sender-to-receiver words in conversation?

rdt3.0: channels with errors *and* loss

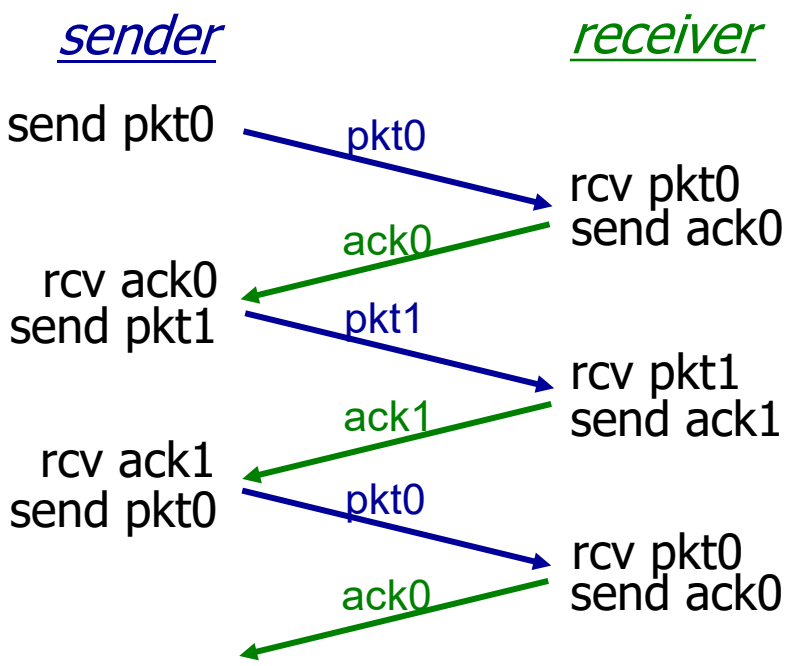
Approach: sender waits "reasonable" amount of time for ACK

- retransmits if no ACK received in this time
- if pkt (or ACK) just delayed (not lost):
  - retransmission will be duplicate, but seq #s already handles this!
  - receiver must specify seq # of packet being ACKed
- ***General Name:*** Automatic Repeat Request (ARQ)

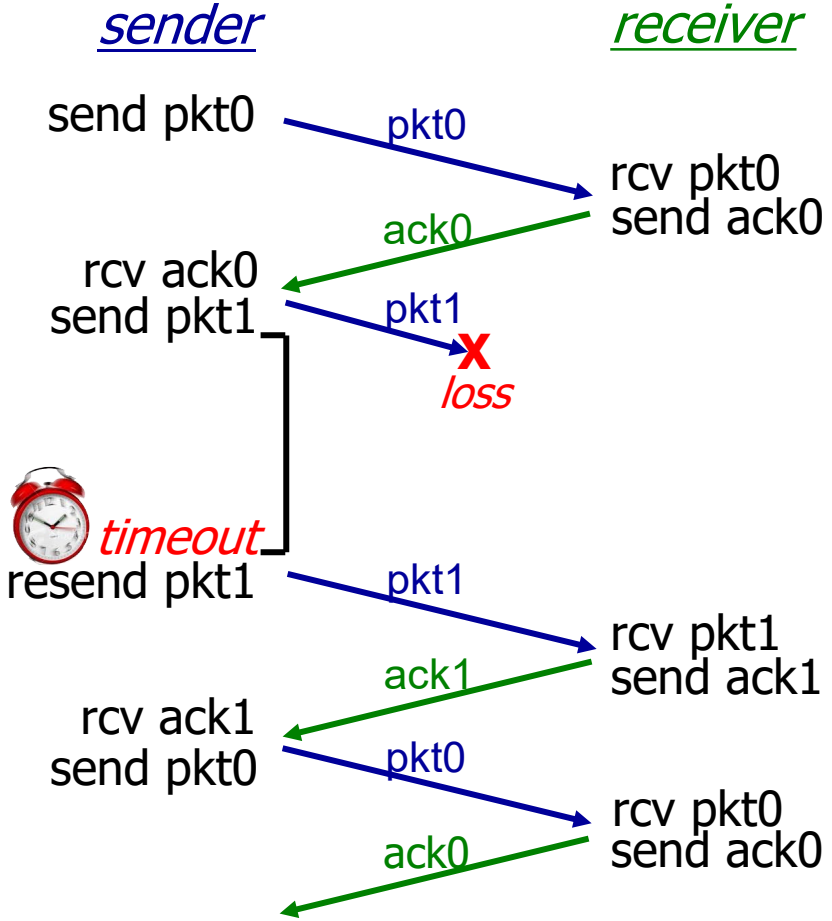


timeout

rdt3.0 in action

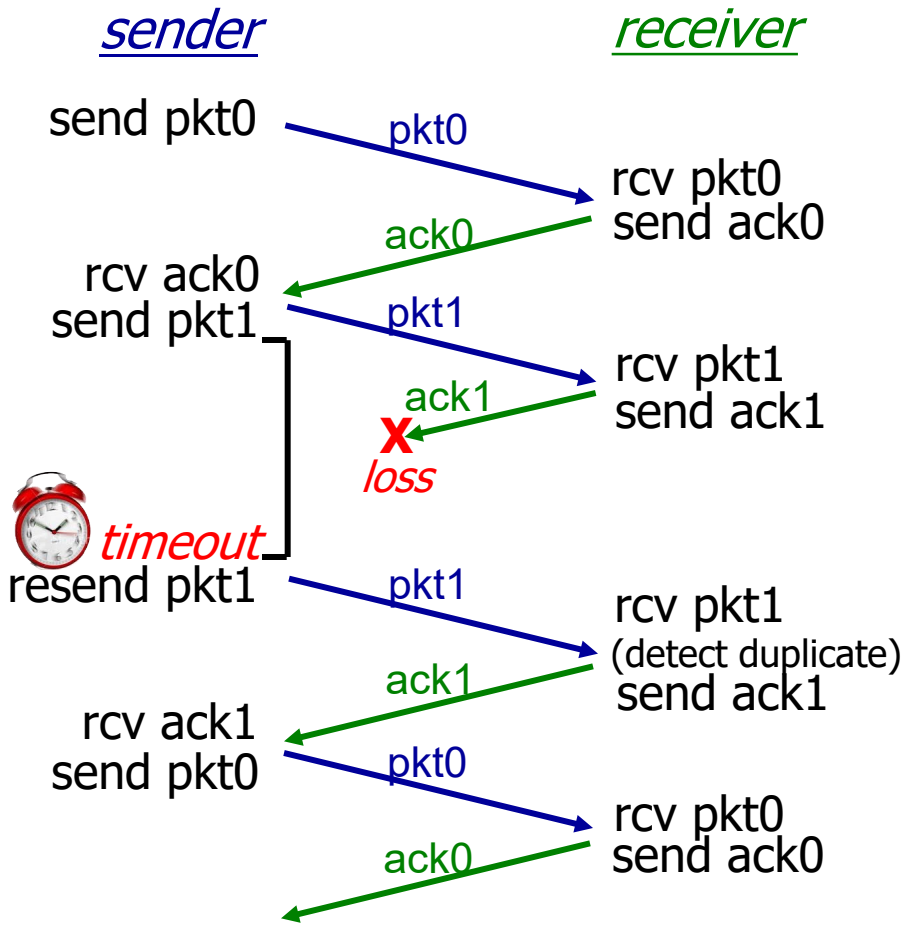


(a) no loss

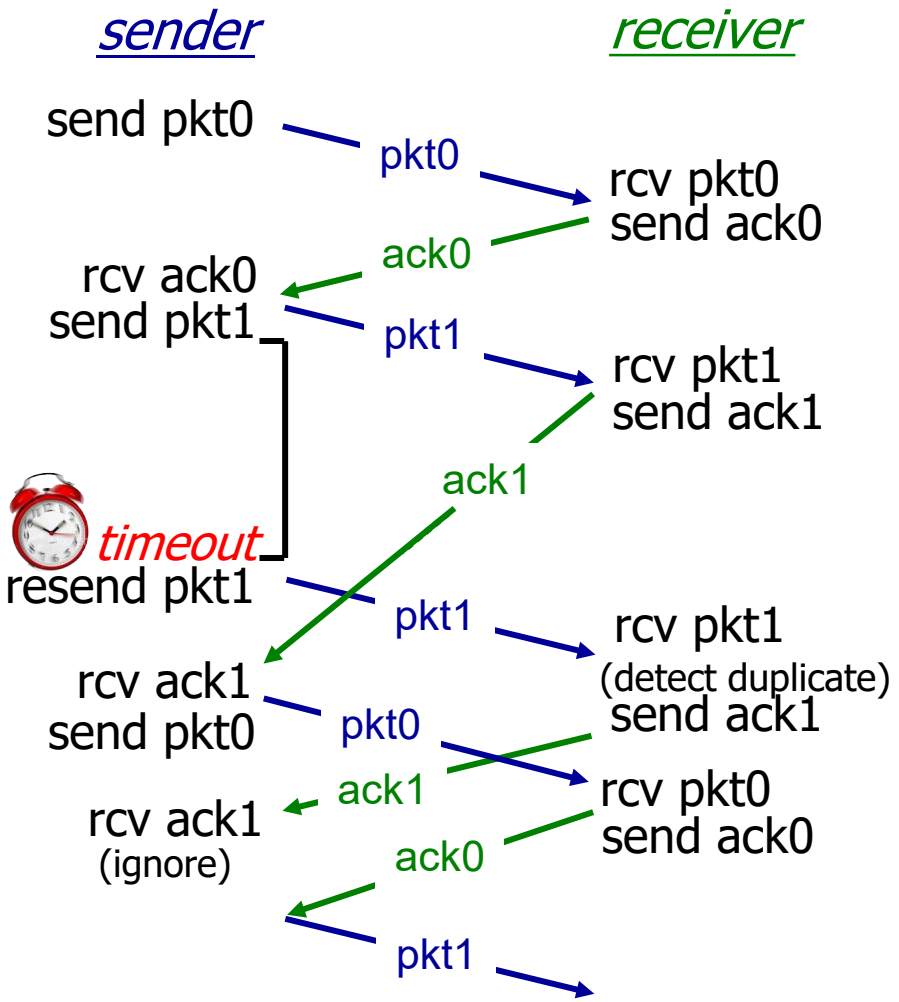


(b) packet loss

rdt3.0 in action

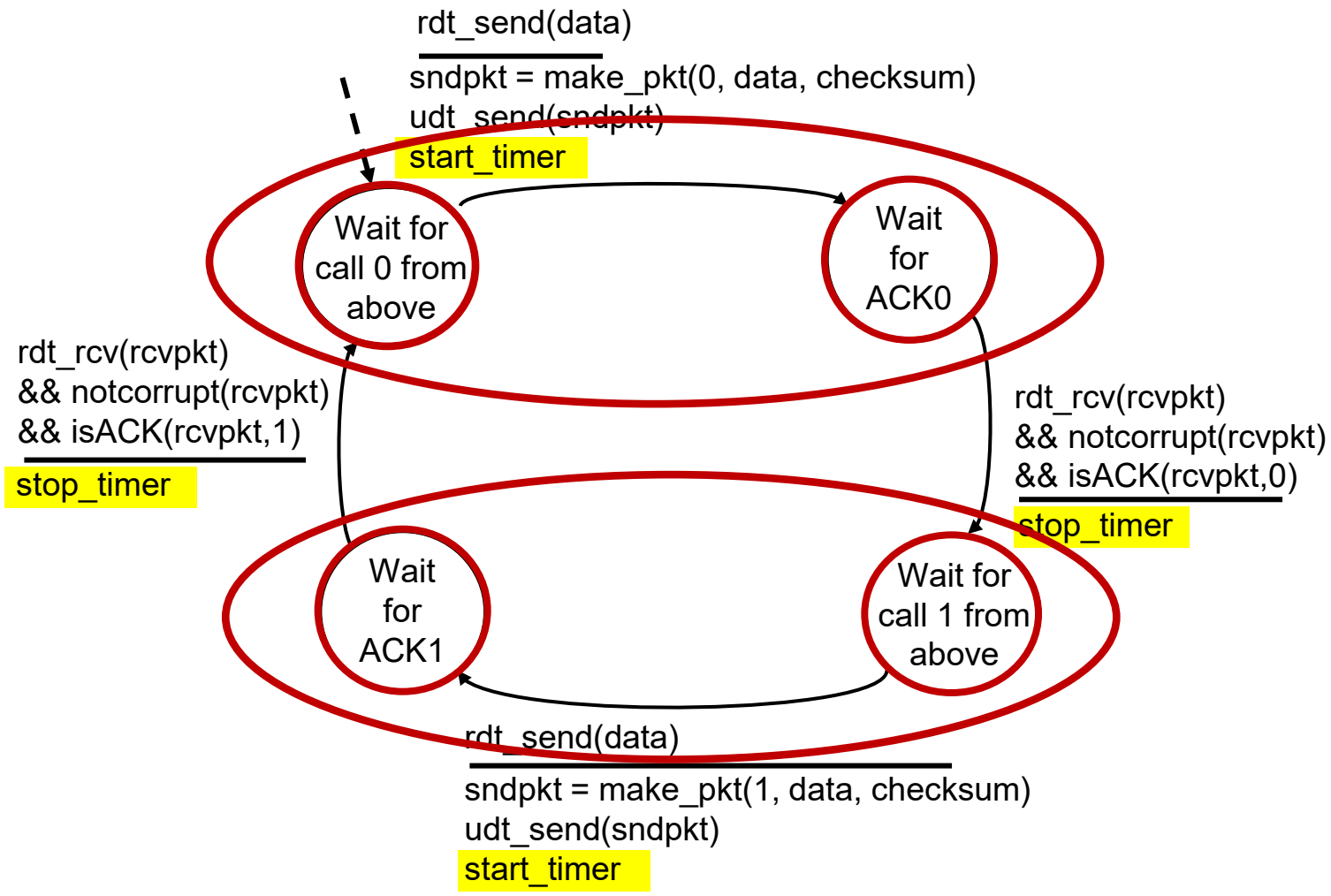


(c) ACK loss

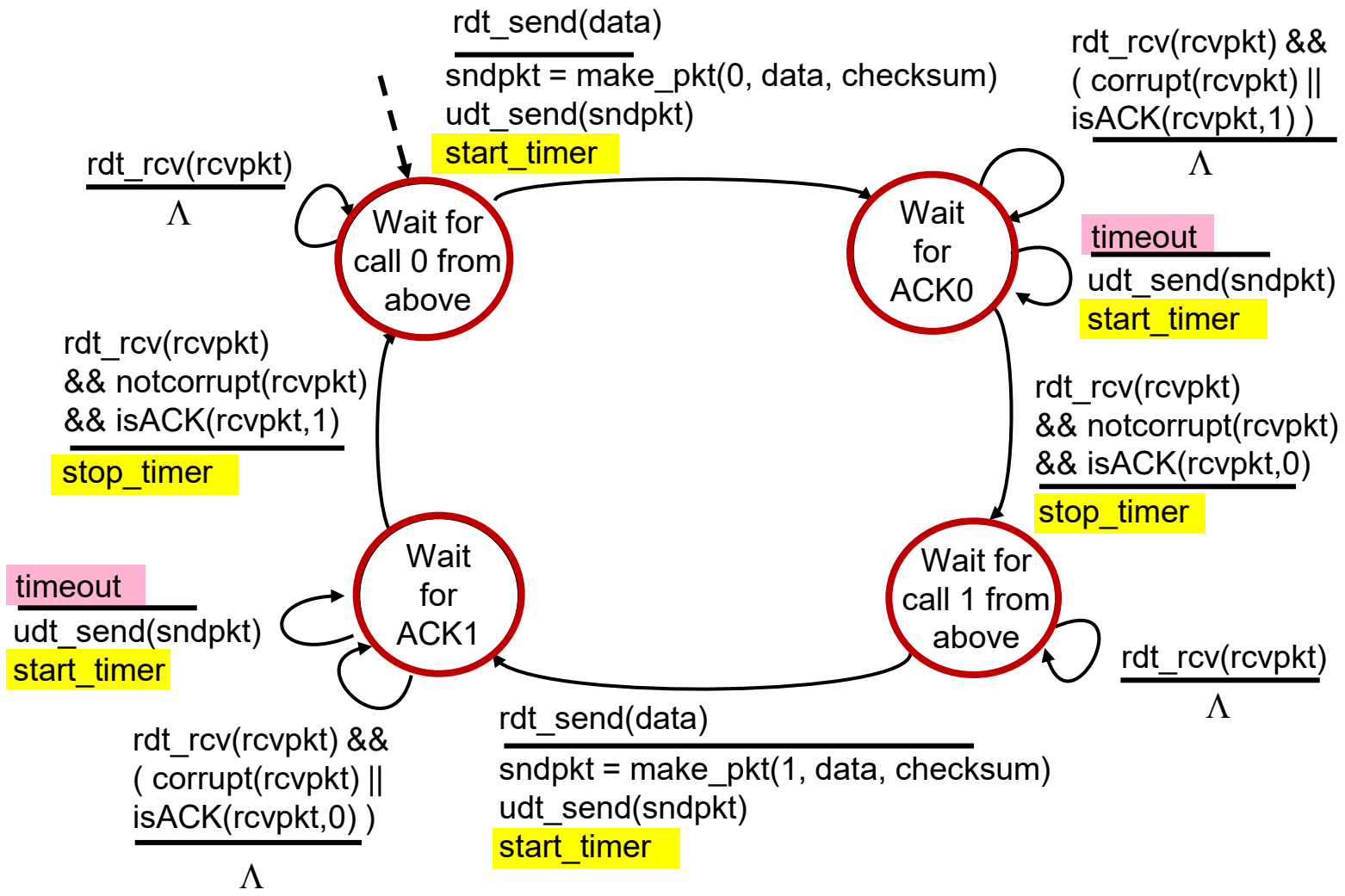


(d) premature timeout/ delayed ACK

rdt3.0 sender



rdt3.0 sender



Q1: Reliable data transfer

- Which of the following are needed for reliable data transfer with only packet corruption (and no loss)? Use only as much as is strictly needed.
  A. Checksums
  B. Checksums, ACKs, NACKs
  C. Checksums, ACKs
  D. Checksums, ACKs, sequence numbers
  E. Checksums, ACKs, NACKs, sequence numbers

Q2: Reliable data transfer

- Which of the following are needed for reliable data transfer to handle packet corruption and loss? Use only as much as is strictly needed.
  A. Checksums, timeouts
  B. Checksums, ACKs, sequence numbers
  C. Checksums, ACKs, timeouts, pipelined protocol
  D. Checksums, ACKs, sequence numbers, timeouts
  E. Checksums, ACKs, NACKs, sequence numbers, timeouts